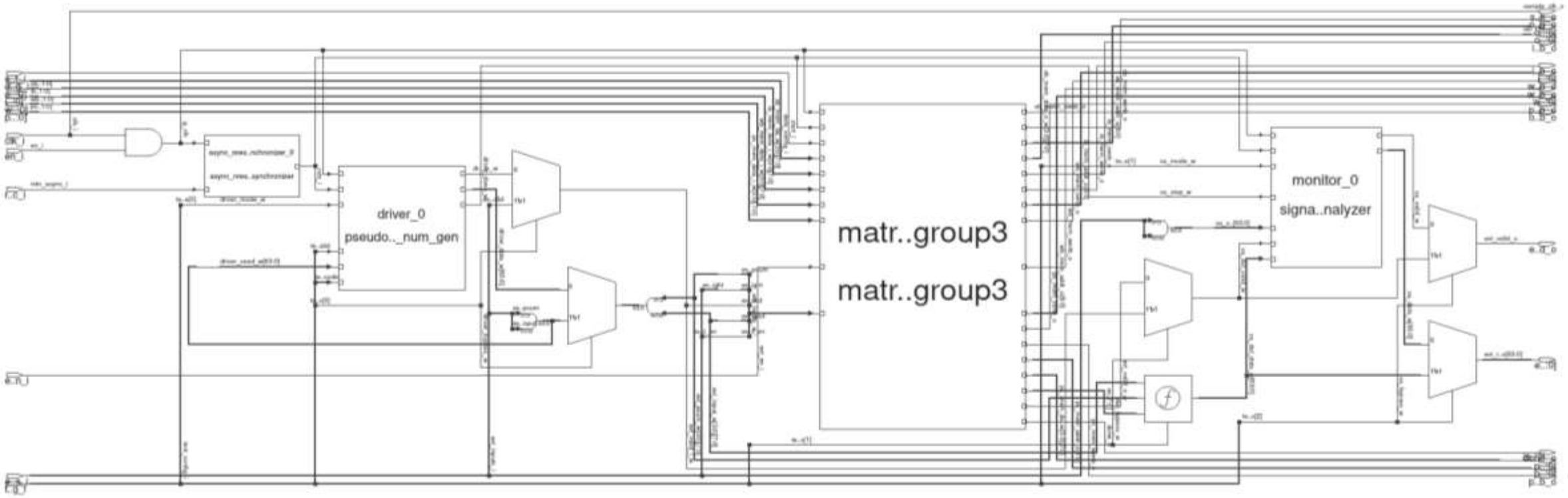
Design Review

Group 3
Rui Wang, Leyang Huang, Ashiqur Rasul, Andrey Li

Georgia Tech

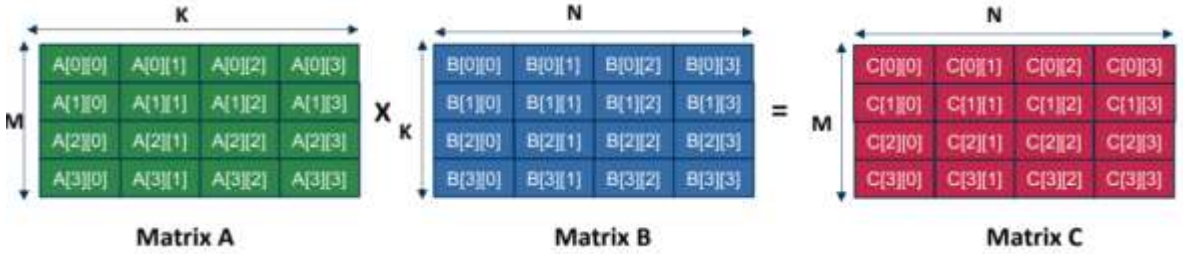
RTL Status

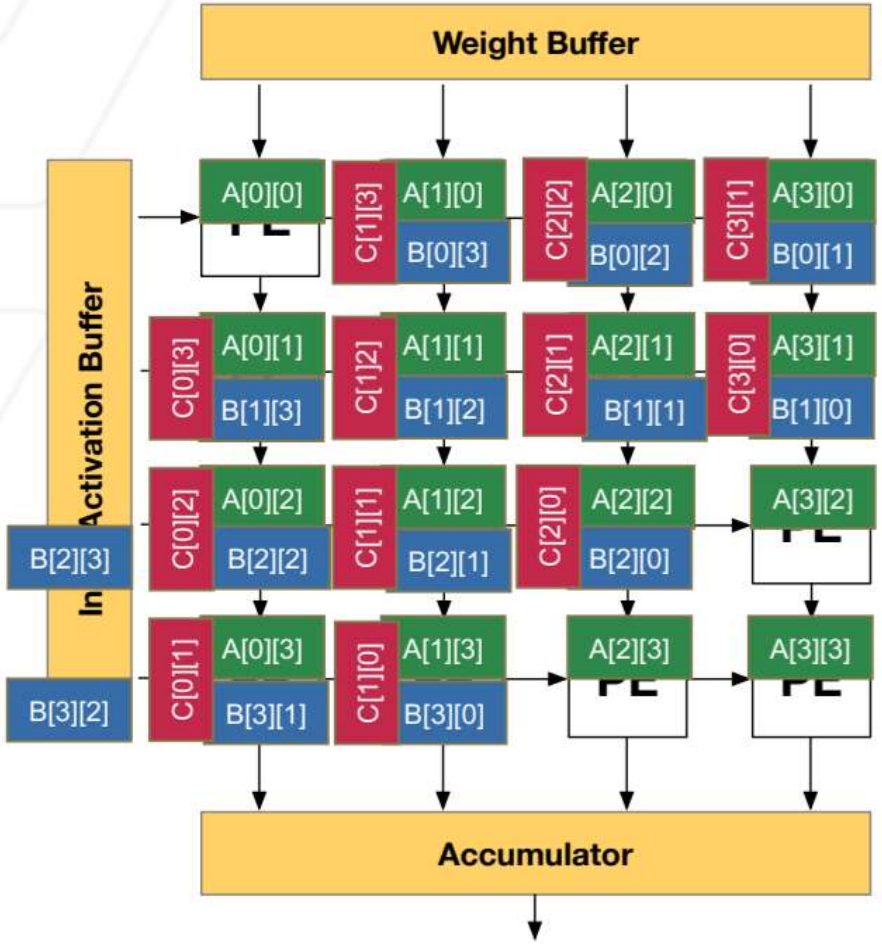
- Completed basic operation and integrated with wrapper
- Supports external, memory, and BiST modes
- Parametrizable matrix sizes



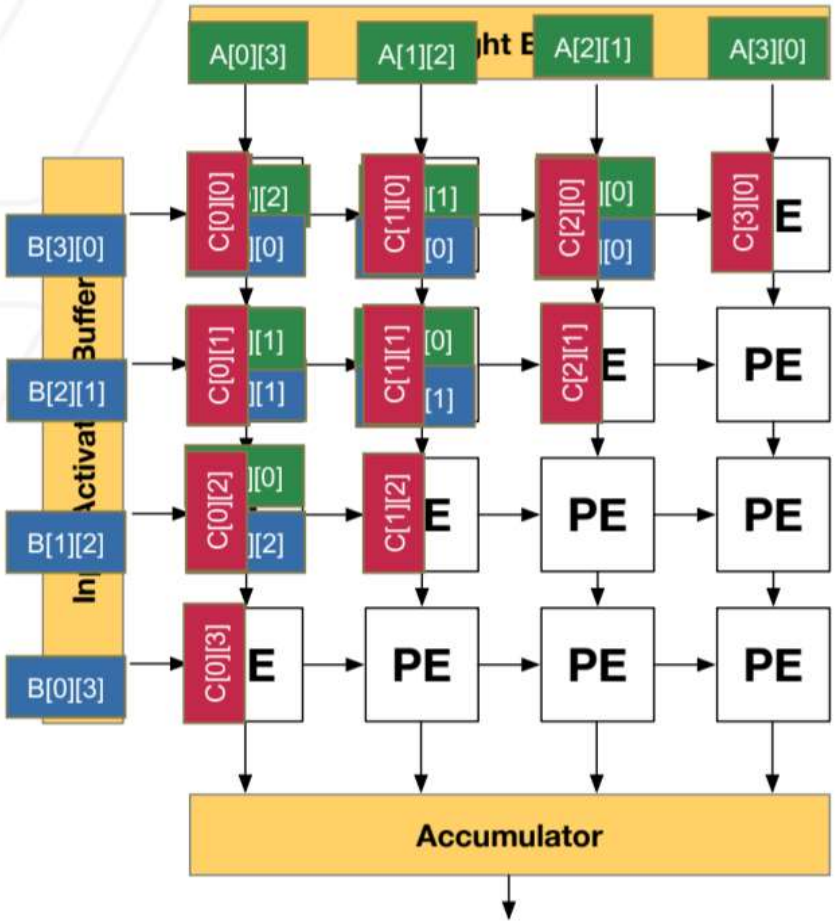
Georgia Tech

Output vs. Weight Stationary





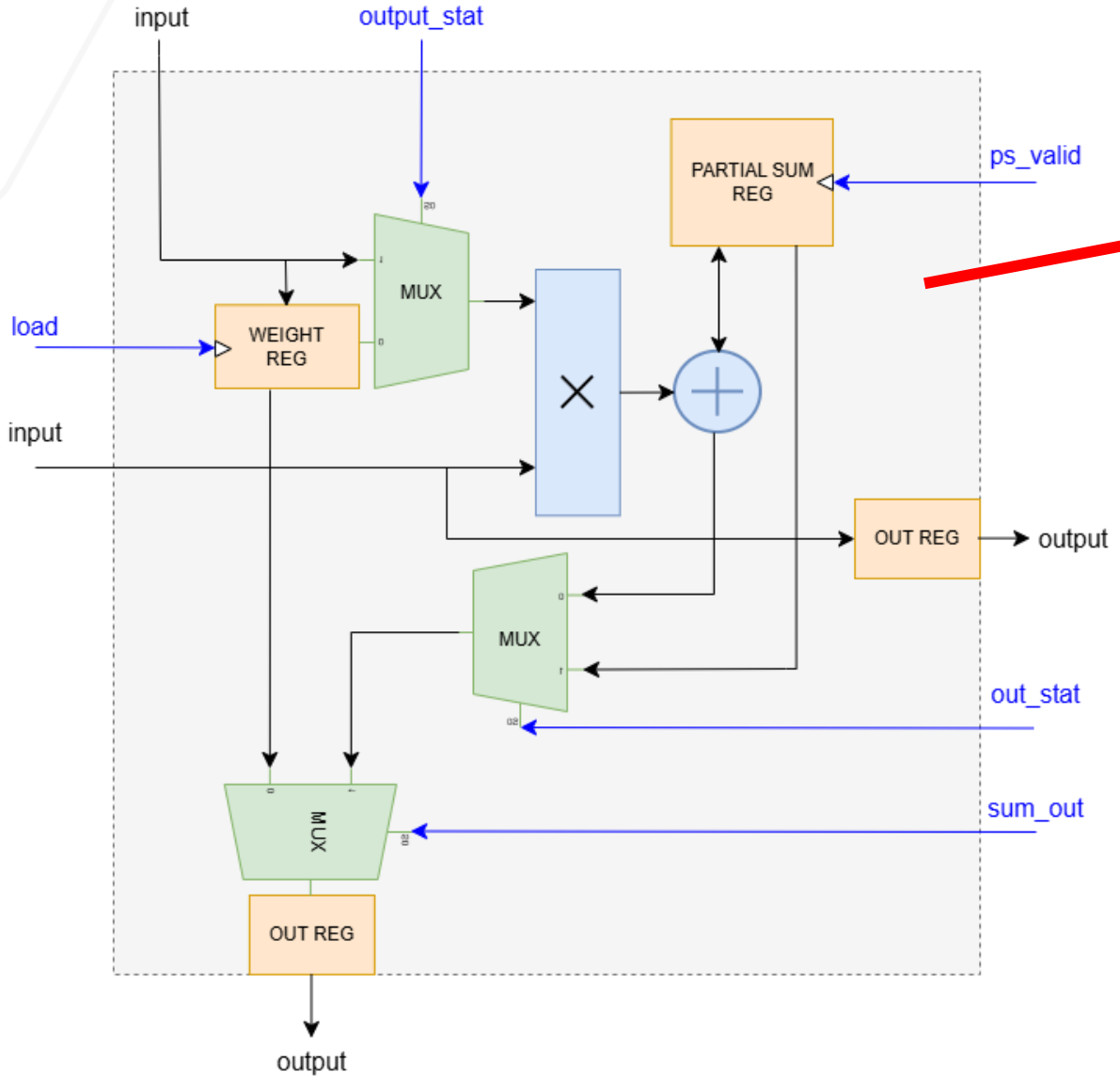
VS



Slide Courtesy: Dr. Tushar Krishna (GT ECE – Synergy Lab)

Georgia Tech

Systolic Array



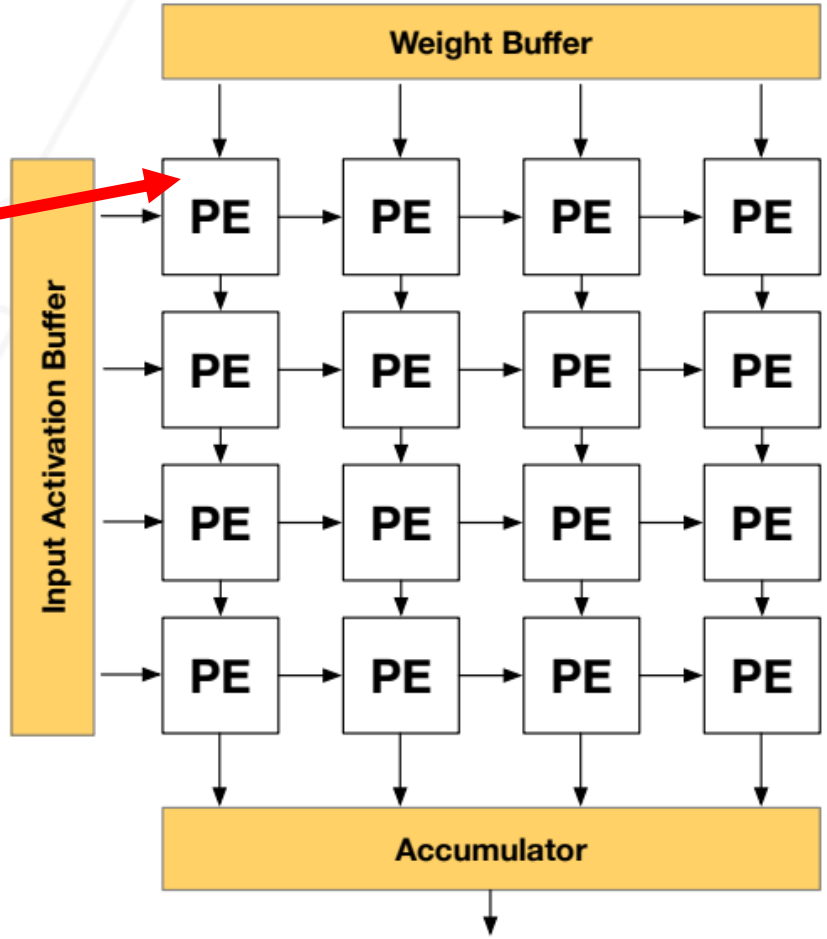


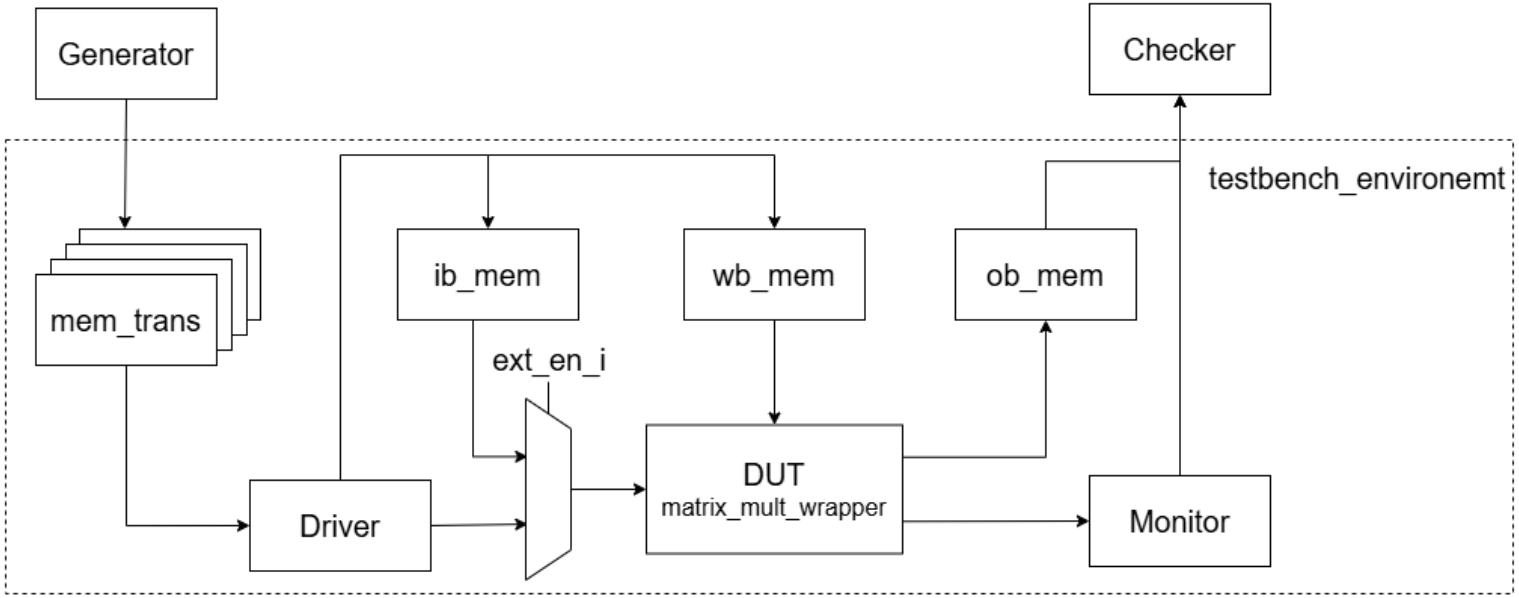
Figure Courtesy: Dr. Tushar Krishna (GT ECE – Synergy Lab)

Verification Status

Level	Directed	Edge-case	Random	Coverage
Processing Element	Passed	Zero, Sat	Passed	Code (84.96%)
Systolic Array	Passed	Zero, Sat	Passed	----
Matrix Mult (w/ Memory)	Passed	Zero, Sat	Passed	Code (84.56%)
Wrapper (External)	Passed	Zero, Sat	Passed	Code (47.93%)
Wrapper (Memory)	Passed	Zero, Sat	Passed	Code (35.46%)
Wrapper (BiST)	Passed	In progress	In progress	In progress

Verification Plan

- Directed test ✅
  - Inputs.txt, Weights.txt, Outputs.txt
- Random test ✅
  - generate_golden.py
  - verify_output.py
- Edge cases ✅
  - zero, full, saturation, negative
- Improving code coverage
  - different random seeds
- Add functional coverage
  - Coverage group, balanced bins
- Add assertions
  - Immediate assertions, concurrent assertions



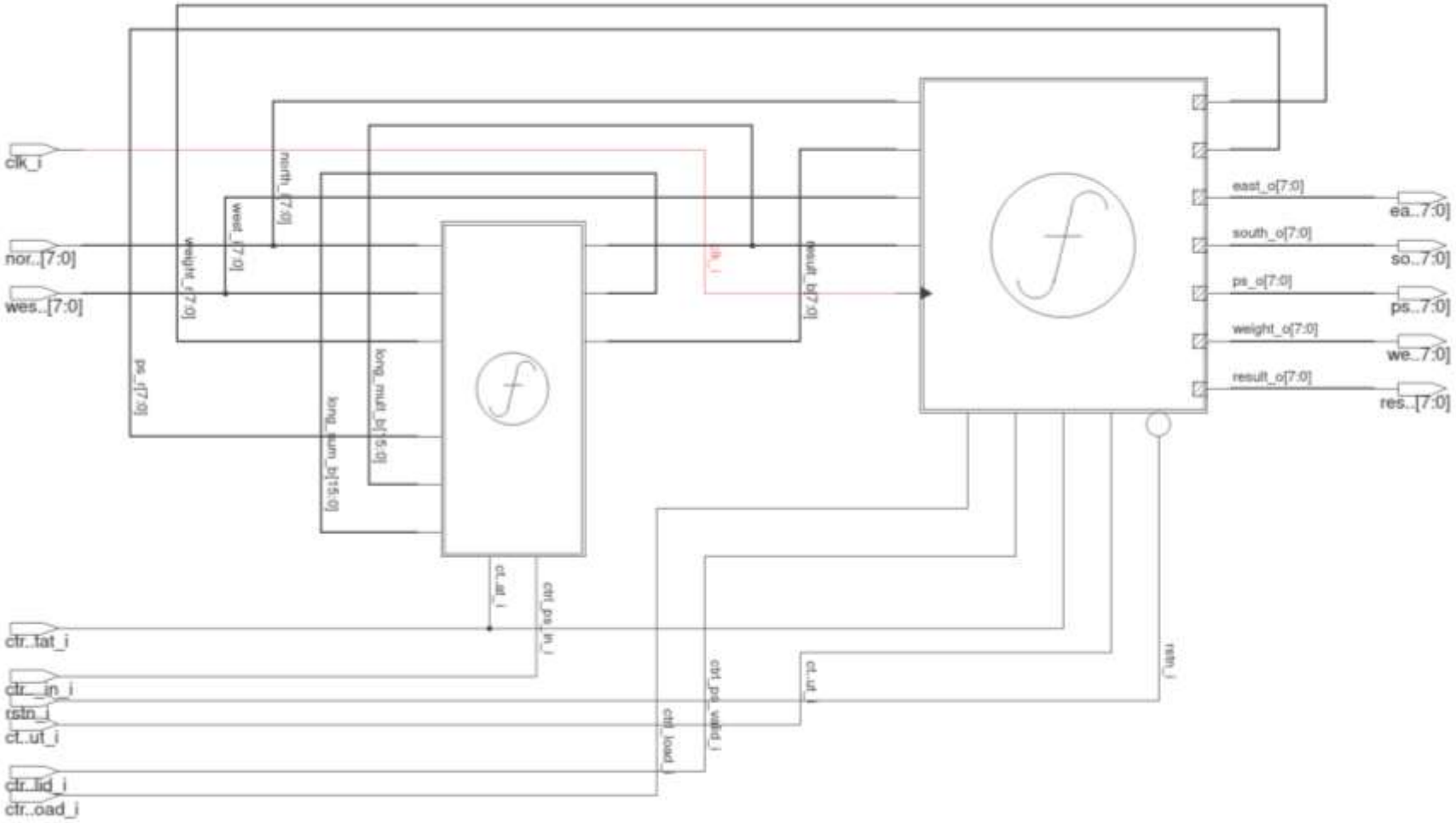
RTL Quality

- Best practices to ensure synthesizable RTL (checklist)

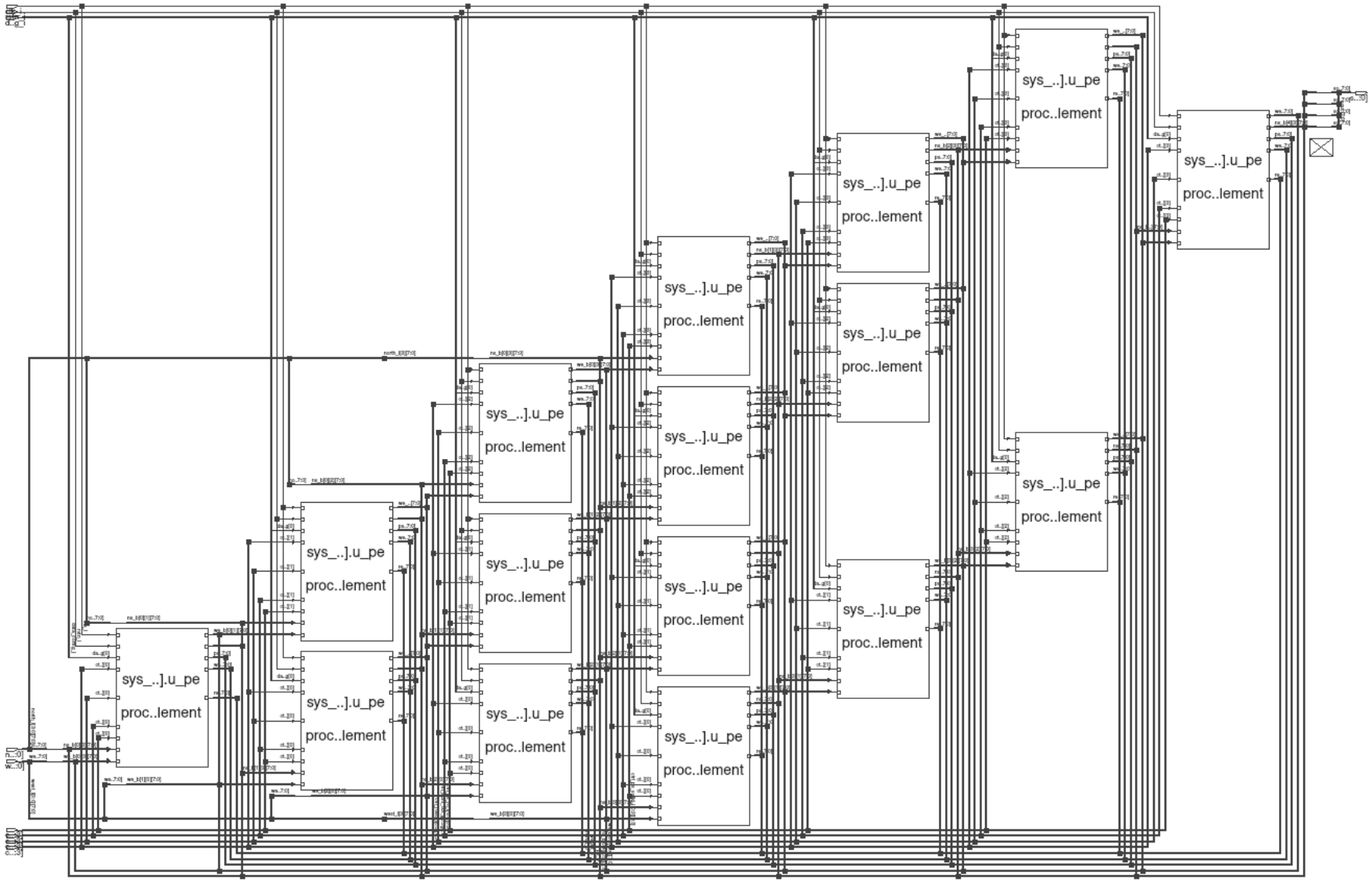
Best Practices	Check
Use of always_ff, always_comb, and always_latch only	✅
Proper use of case statements (No unassigned LHS variables)	✅
Did not mix blocking and non-blocking assignments	✅
Use of generate for loops with genvars	✅
Parameterizable systolic array dimensions and data width	✅
Use of immediate assertions	❌
No combinational logic through clock path (only ICGs)	✅
Proper use of Mux-D statements for clock gating inferencing	❌

Backup Slides

Processing Element

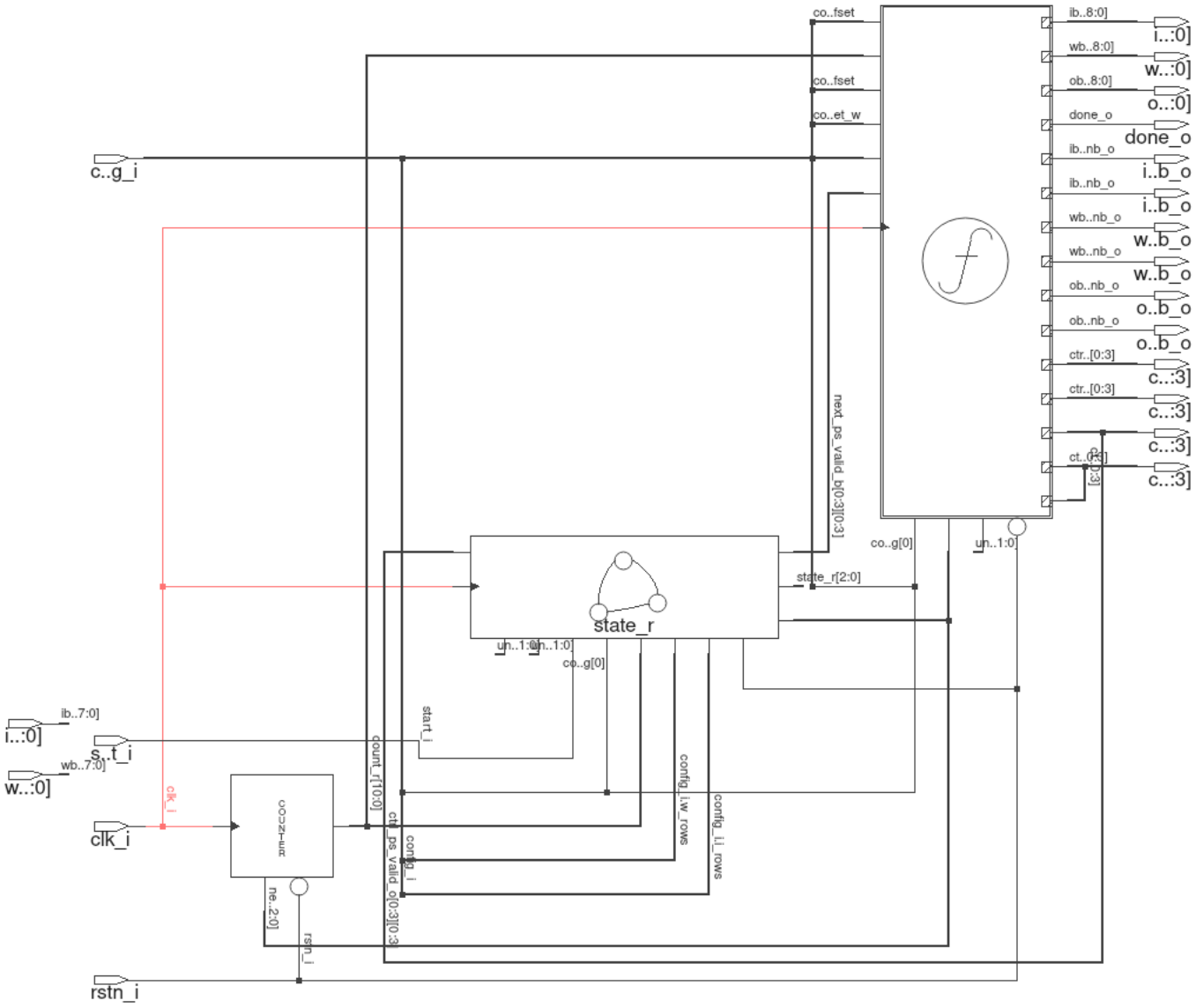


Systolic Array



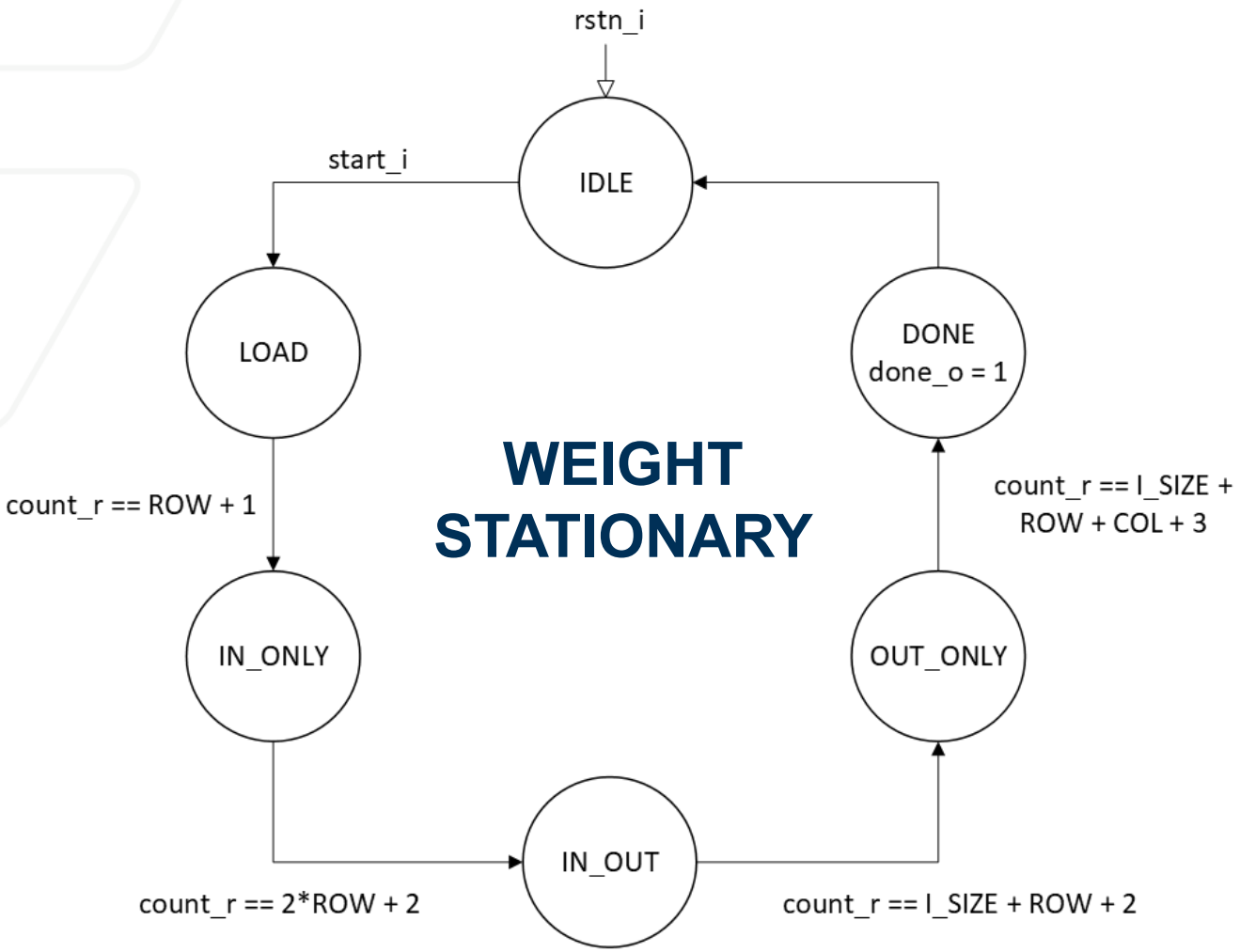
Georgia Tech

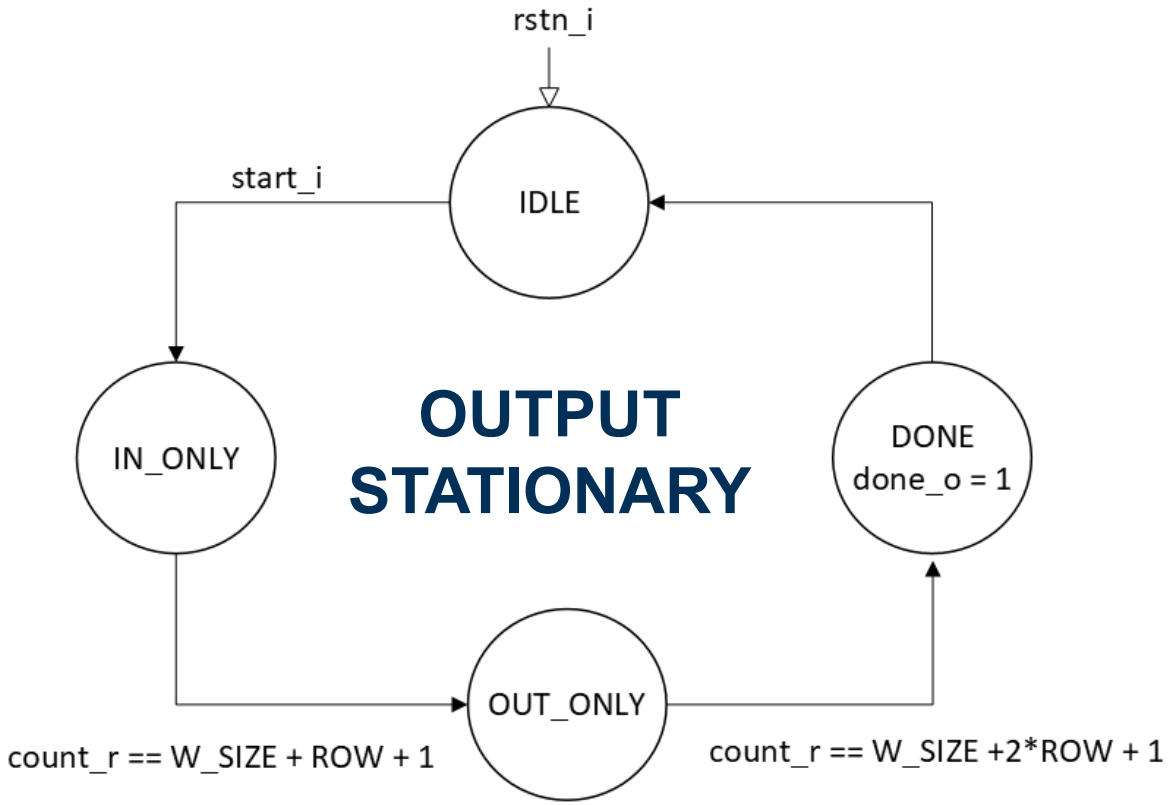
Controller


co..fset
co..fset
co..et_w
c..g_i
ib..8:0]
wb..8:0]
ob..8:0]
done_o
ib..nb_o
ib..nb_o
wb..nb_o
wb..nb_o
ob..nb_o
ob..nb_o
ctr..[0:3]
ctr..[0:3]
i..:0]
w..:0]
o..:0]
done_o
i..b_o
i..b_o
w..b_o
w..b_o
o..b_o
o..b_o
c..:3]
c..:3]
c..:3]
c..:3]
next_ps_valid_b[0:3][0:3]
co..g[0]
un..1:0]
state_r[2:0]
state_r
un..1:0]
un..1:0]
co..g[0]
start_i
count_r[10:0]
config_i_w_rows
config_i_i_rows
ctl_ps_valid_o[0:3][0:3]
config_i
ib..7:0]
i..:0]
s..t_i
wb..7:0]
w..:0]
clk_i
clk_i
rst_i
rst_i
rstn_i

Controller (Memory Mode)

- Based on internal counter that increments during active operation





Verification Approach

- Constructs using in our verification
  - Classes: mem_trans
  - Tasks: memory_mode(), external_mode(), bist_mode()
  - Functions: read_mem_files()
  - Rand: $urandom_range(), np.random()
- Regression flow
  - run.sh
    - generate_golden.py; randomly generate random memory maps
    - make vcs; simulate and save results
    - verify_output.py; compare results with golden memory
    - make urg; extract coverage report
- High-level simulator for golden values
  - Milestone 1a, the systolic array with saturation simulator

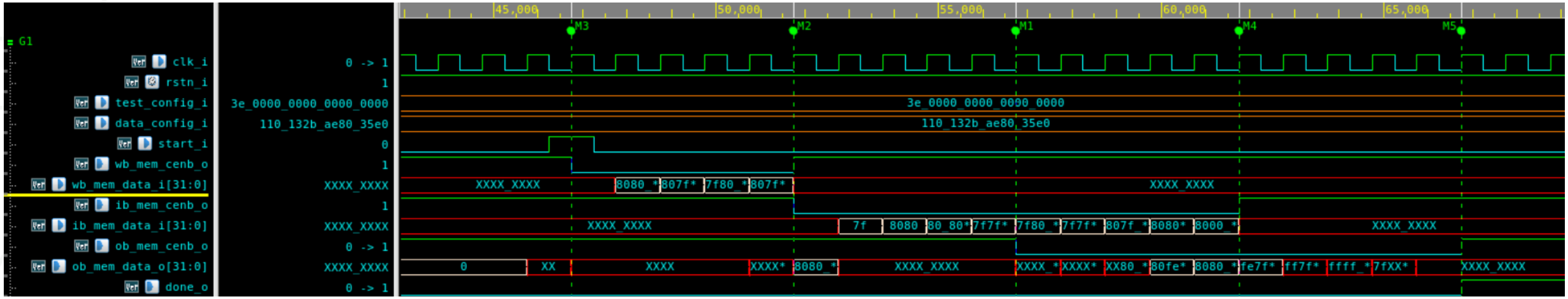
Verification Results

```
@0: testname = default
@9750===============Memory Mode==================
@9750: mem input_trans: 0000007f 00008080 0080807f 7f7f7f80 7f80807f 7f7f807f 807f7f00 80800000 80000000
@9750: mem weight_trans: 8080807f 807f807f 7f808080 807f7f7f
==========Initial Memory==========
@45750: ib_mem.data: 349: 0000007f 350: 00008080 351: 0080807f 352: 7f7f7f80 353: 7f80807f 354: 7f7f807f 355: 807f7f00 356: 80800000 357: 80000000
@45750: wb_mem.data: 299: 8080807f 300: 807f807f 301: 7f808080 302: 807f7f7f
@45750: ob_mem.data: 431: 00000000 432: 00000000 433: 00000000 434: 00000000 435: 00000000 436: 00000000 437: 00000000 438: 00000000 439: 00000000
==========Start Computing==========
==========Computation Finished==========
@66750: ob_mem.data: 431: xxxxxx7e 432: xxxxff7f 433: xx8080ff 434: 80fefefe 435: 8080ffff 436: fe7f7e80 437: ff7f7fxx 438: ffffxxxx 439: 7fxxxxxx
@66750===============External Mode==================
@66750: mem input_trans: 0000007f 00008080 0080807f 7f7f7f80 7f80807f 7f7f807f 807f7f00 80800000 80000000
@66750: mem weight_trans: 8080807f 807f807f 7f808080 807f7f7f
@88750: ext_result_o = 000000000000007e
@89750: ext_result_o = 000000000000ff7f
@90750: ext_result_o = 00000000008080ff
@91750: ext_result_o = 0000000080fefefe
@92750: ext_result_o = 000000008080ffff
@93750: ext_result_o = 00000000fe7f7e80
@94750: ext_result_o = 00000000ff7f7f00
@95750: ext_result_o = 00000000ffff0000
@96750: ext_result_o = 000000007f000000
@96750===============BiST Mode==================
@96750: mem weight_trans: 8080807f 807f807f 7f808080 807f7f7f
@116750: tracking driver_data_w = f0ca684288882469
@117750: tracking driver_data_w = e194d085111048d2
@118750: tracking driver_data_w = c329a10a222091a4
@119750: tracking driver_data_w = 8653421444412348
@120750: tracking driver_data_w = 0ca6842888824690
@121750: tracking driver_data_w = 194d085111048d20
@122750: tracking driver_data_w = 329a10a222091a40
@123750: tracking driver_data_w = 6534214444123481
@123750: ext_valid_o is asserted, ext_result_o = 222091a0bc5522f2
$finish called from file "./tb_matrix_mult.sv", line 233.
```

matrix_mult_wrapper.log

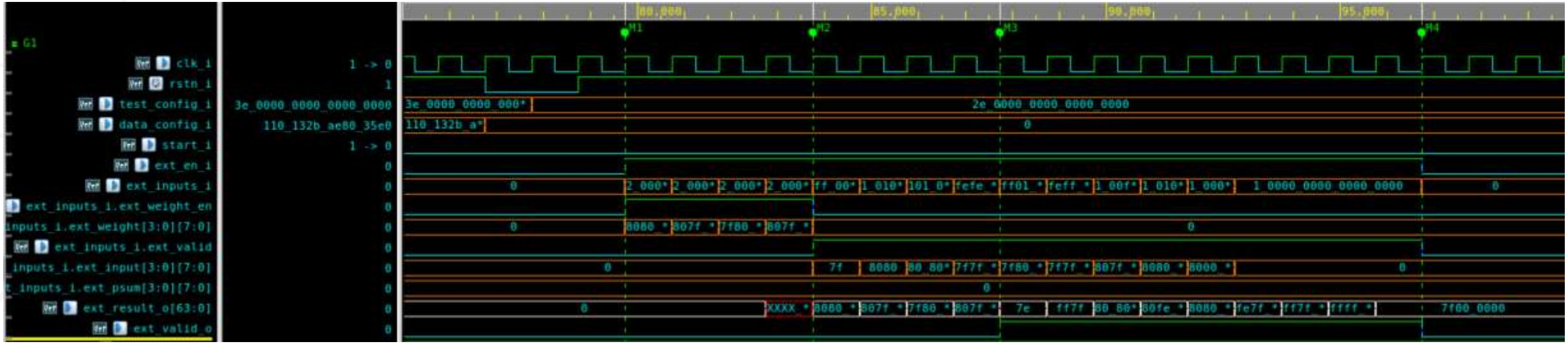
Georgia Tech

Verification Results (cont.)



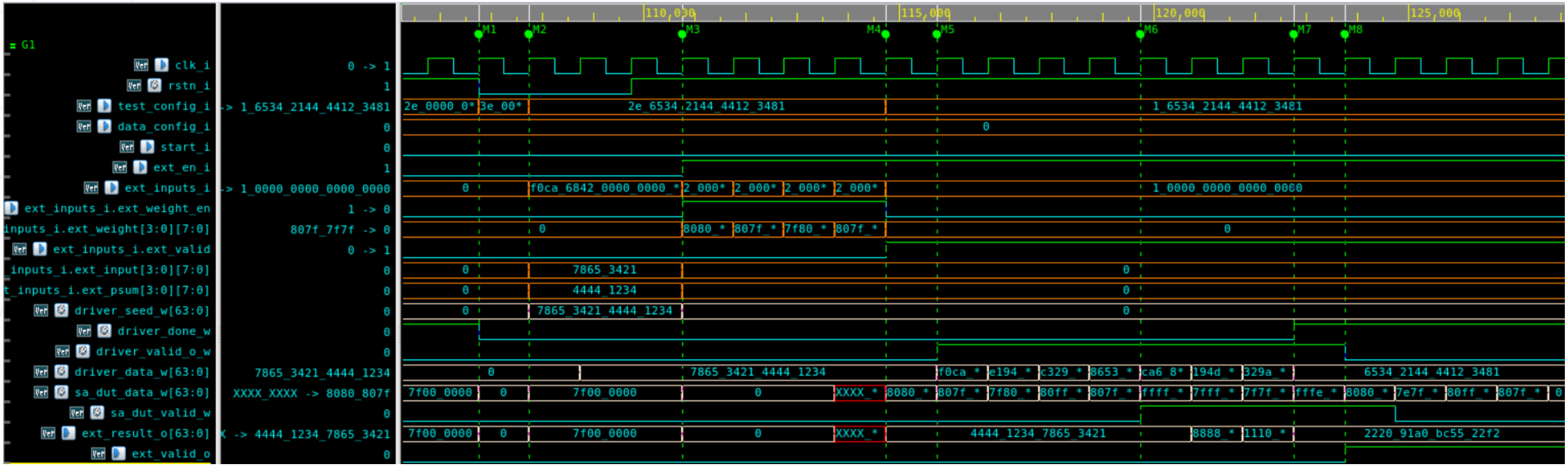
Memory Mode

Verification Results (cont.)



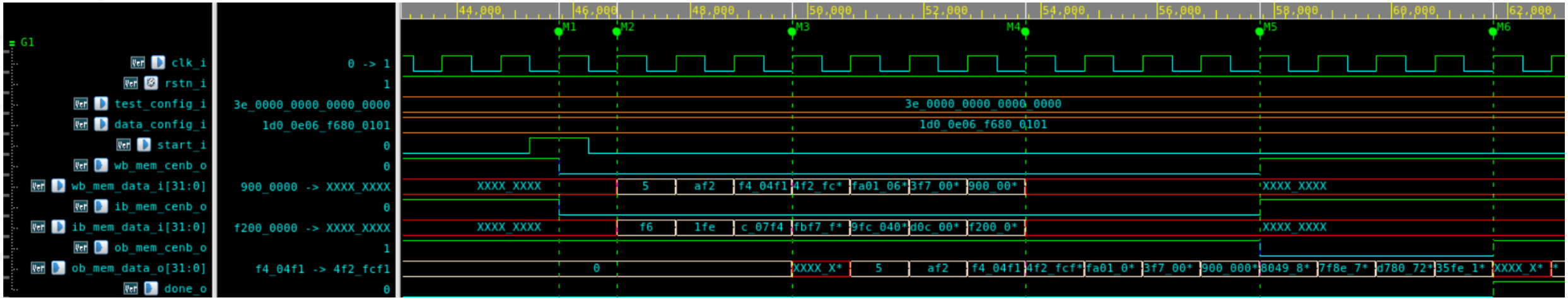
External Mode

Verification Results (cont.)



BiST mode

Verification Results (cont.)



Output Stationary

Specifications (Screenshot)

Feature	Description
Number of rows	4 (Programable by Designer)
Number of columns	4 (Programable by Designer)
Input data format	Signed Integer (2's Complement)
Input data width	8 bits
Output data width	8 bits
Handling data overflow	Saturation
Modes supported	Memory, External, & BiST
IO ports	See matrix_mult.sv
Reset	Active-Low Reset (Reset when 0)
Process node	TSMC 65GP
Timing model	NLDM
Power supply	0.8 ~ 1 V
Target corner	TT
Minimum clock frequency	100 Mhz
Highest metal allowed	M7
Target Area	172.8 um x 172.8 um

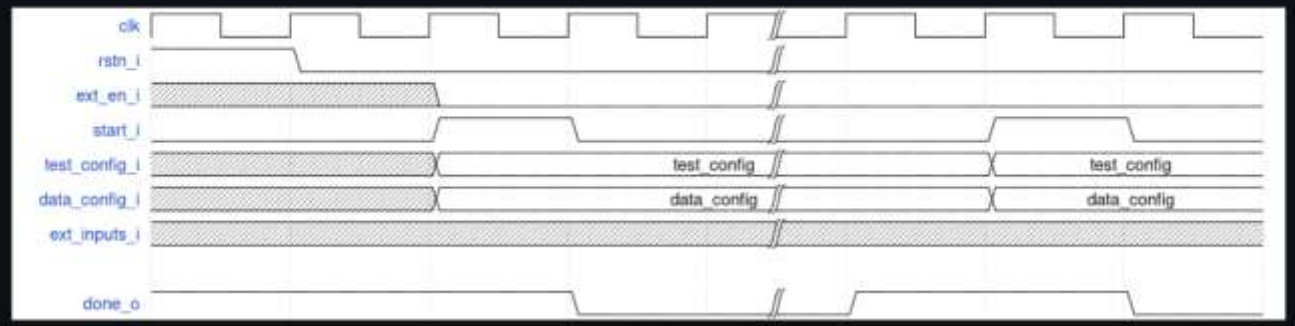
Georgia Tech

Memory Mode

1. Memory mode: The memories are loaded and available. Your internal controller should load the weights into the array, process all inputs and partial sums, and save the results in the output memory. The input test configuration and data configuration are fed to your Matrix Multiplication module, as well as the start bit. After processing, the done signal indicates completion, and the results are written to the output memory.

   Due to the nature of our testing infrastructure, the start condition of your internal controller must be at the **posedge** of the start_i signal (note that this is different from reading a high start_i value). You can assume that no other posedge will show during the computation. The start_i signal might remain high for an indefinite amount of time (relative to your module's computation time).

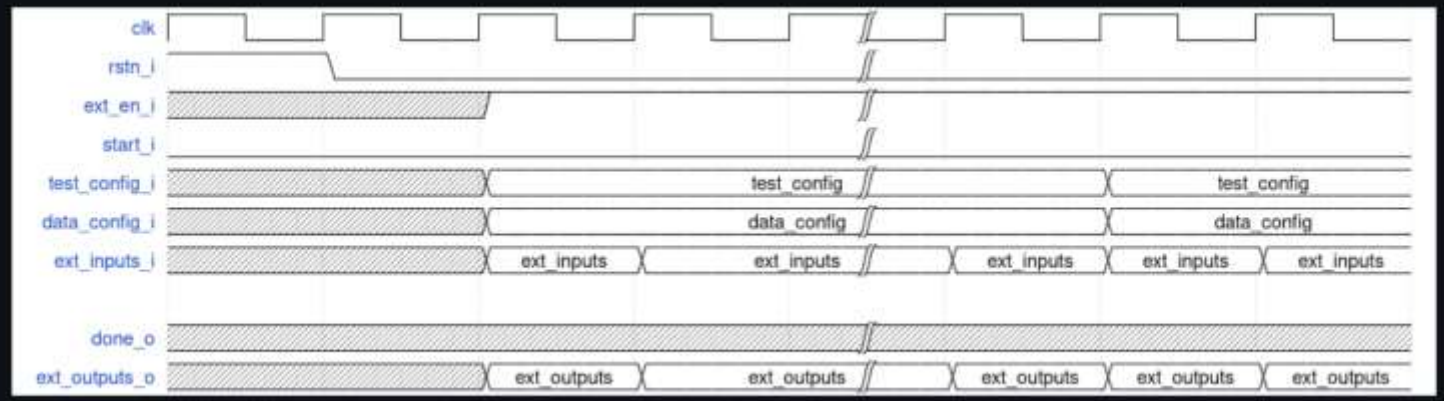
   Shown below is an example of the input and expected output pattern. Assume that the input, weights, and partial-sum memories have already been loaded.



External Mode

2. External mode: The memories are not available, and all signals come from external sources that are provided by the scan chain. In this mode, the matrix multiplication module functions purely as a processing unit systolic array. All inputs, internal control signals, and outputs are handled through the use of input test configurations (test_config_struct) and external signal ports (external_inputs_struct). During external mode you can assume to have full control of your clock signal (you choose how and when to toggle it).

Shown below is an example of the input and expected output pattern. Note that ext_en_i is used to signal our module that External mode has been selected. Weights and input activations are manually loaded into the array row-by-row cycle by cycle through the use of the given external signal ports and a by walking the clock.



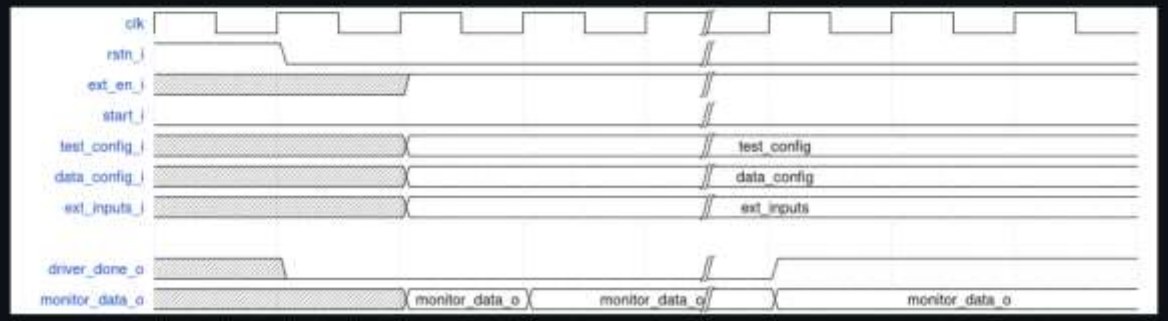
Built-in-Self-Test (BiST) Mode

3. BiST mode: This mode revolves around using the Pseudo Random Number Generator (a.k.a. LFSR) and Singature Analyzer to do at-speed testing. It overlaps extensively with the External mode. The difference comes in the test configuration (test_config_struct) as now the input activations are generated by the LFSR, streamed into the systolic array, and then compressed on the Signature Analyzer. Once the Signature is ready (which is determined by the LFSR's stop code) it will be used to determine if incorrect values were produced. This mode helps us isolate the design from: (1) the memory interface, and (2) the external signal data ports. This setup allows to test the design at maximum speed.

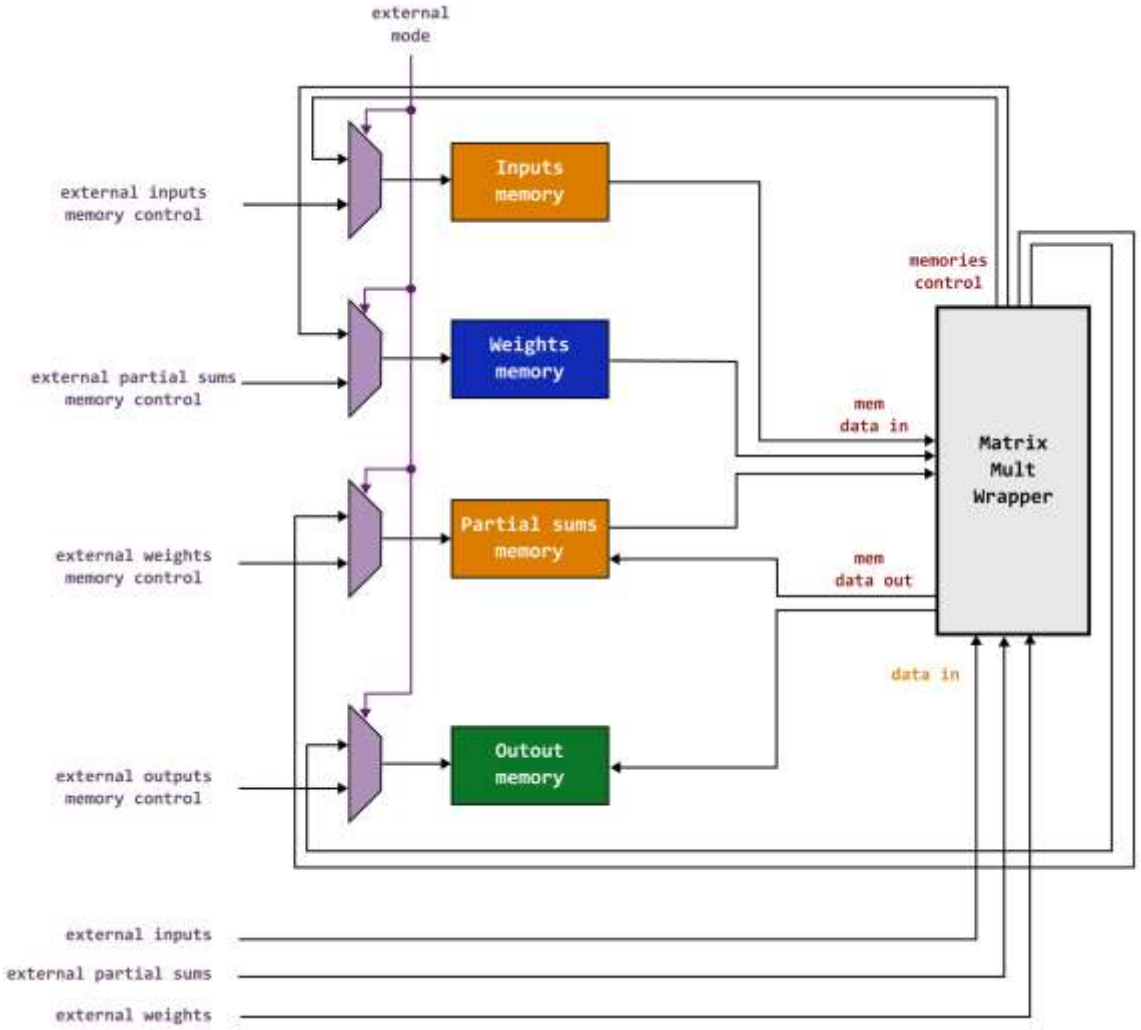
   In this mode:

   i. The external signal ports are leveraged to pre-load the weights into the array (similar to External mode)
   ii. Afterwards, the LFSR is enabled (test_config_struct.driver_valid) and generates pseudo random data that is fed into your design (take careful note of what ports are driven with the LSFR's outputs).
   iii. After the data propagates through your design, it will reach the Signature Analyzer, which will compress valid data cycle by cycle until no more valid data is received. Your module is responsible for receiving the LSFR's valid signal and adequately sending a valid signal to the Signature Analyzer.

   Shown below is an example of the input and expected output pattern.

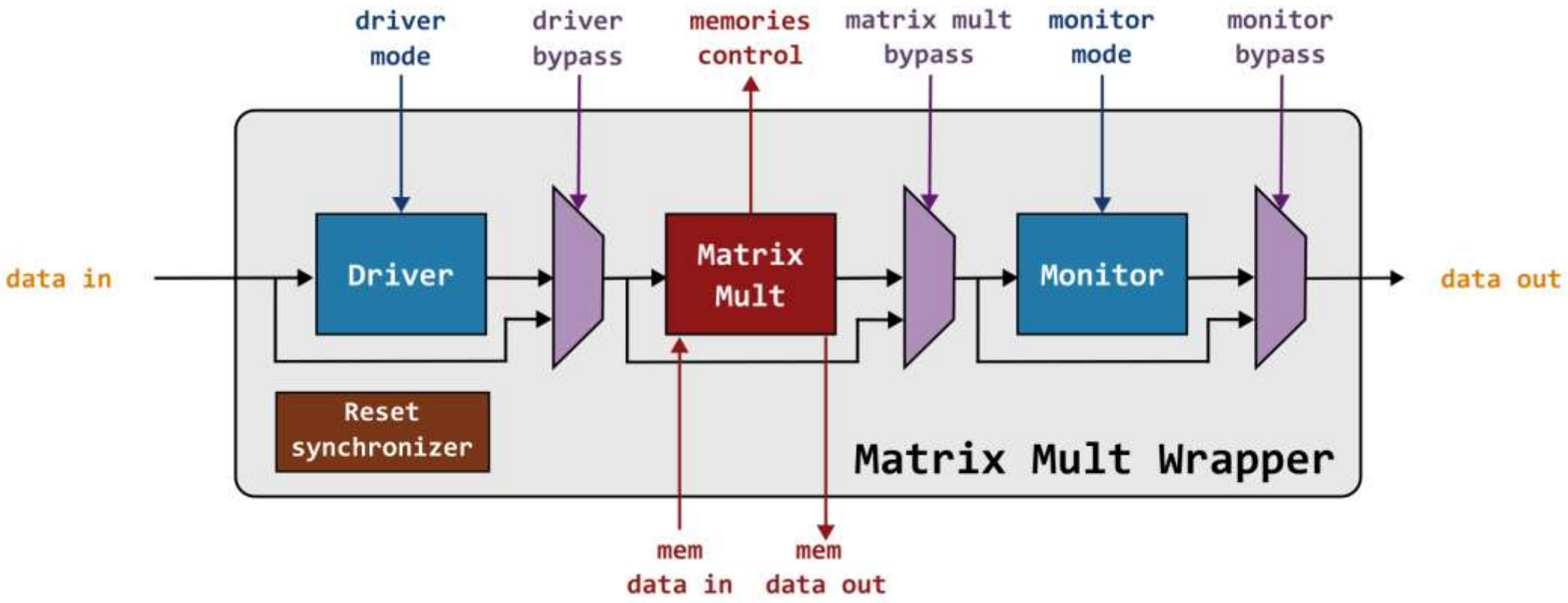


Top Level (Student Facing)

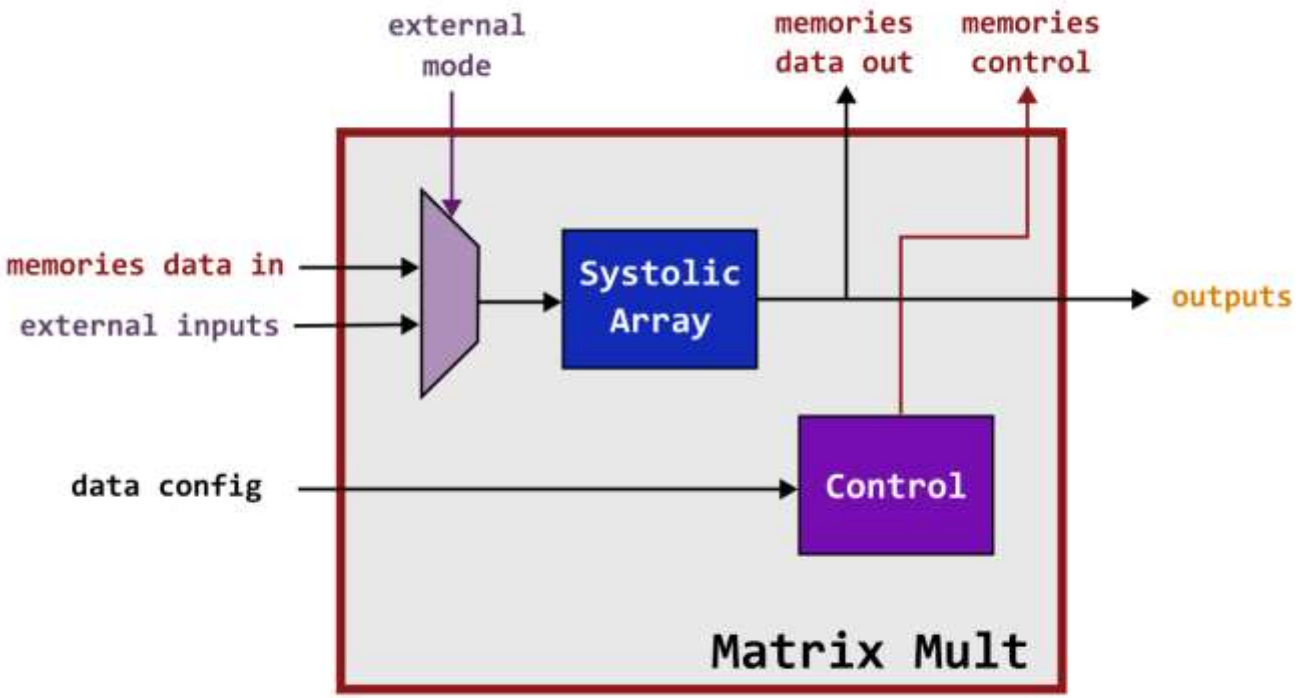
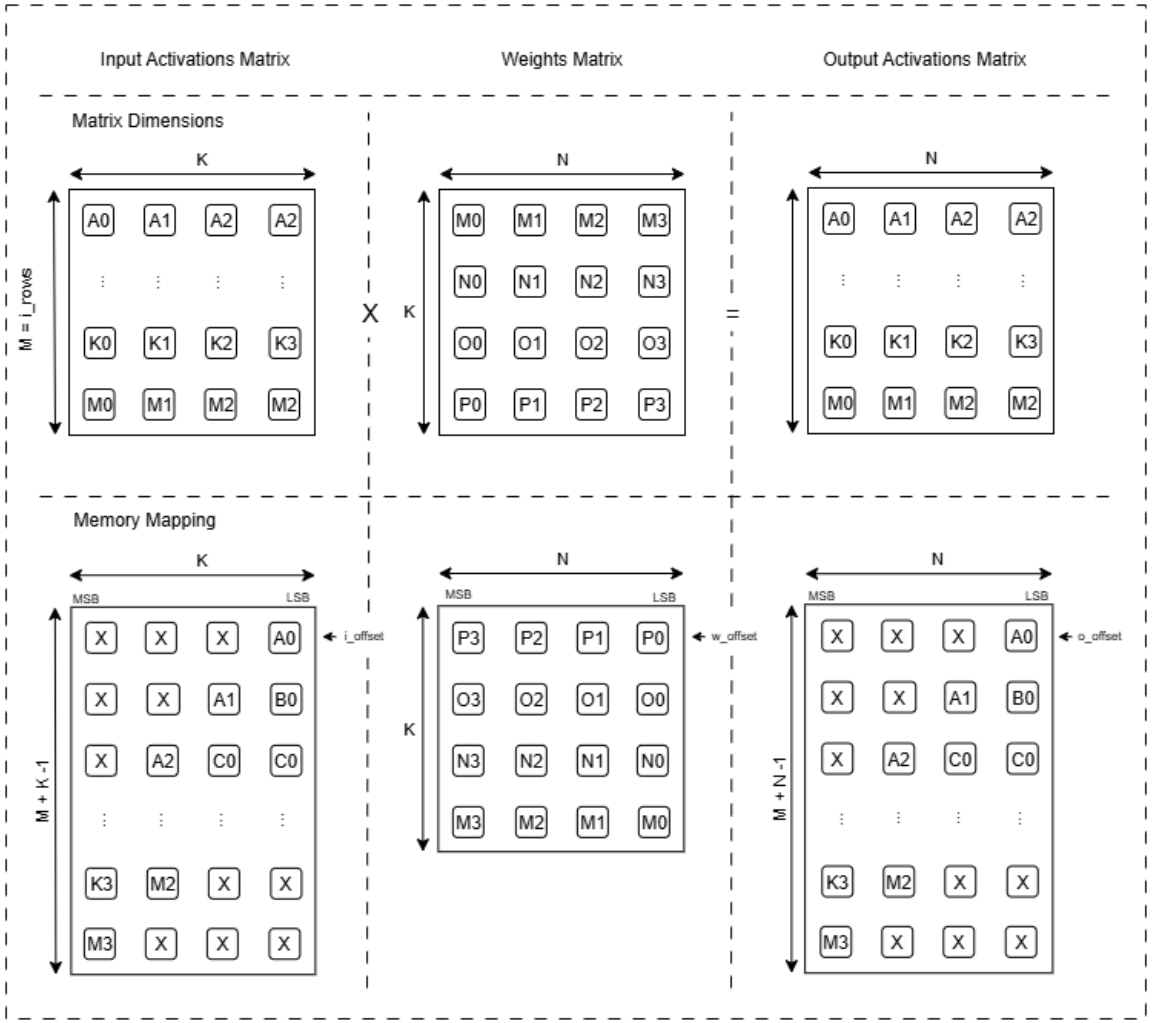


Georgia Tech

Matrix Mult Wrapper



Matrix Mult


external
mode
memories
data out
memories
control
memories data in
external inputs
Systolic
Array
outputs
data config
Control
Matrix Mult

Input Format and Memory Mapping